

Heterogenous Computing Übungsblatt 1

0. Einleitung

Im ersten Teil betrachten wir verschiedene Smart-Home-Architekturen, indem aktuelle Lösungen mit Blick auf Kommunikationsmodelle, Protokolle, Cloud-Anbindung und -Zwang sowie Interoperabilität eingegangen wird.

Auf Grundlage der gesammelten Erkenntnisse wird eine Auswahl für den anschließend entwickelten Prototypen getroffen und begründet.

1. Grundlegende Theorie und Stand der Technik

Das Feld der Smart-Home-System ist breit und bietet neben verschiedenen Ökosystemen auch eine Vielzahl an Herangehensweise in den Bereichen Architektur und Kommunikation, sowohl auf theoretischer Ebene als auch auf Implementationsebene.

1.1. Architekturtypen

Grundlegend sollte zwischen drei Architekturmustern unterschieden werden, die fundamental mit der Verwendung von Cloudtechnologien in Zusammenhang stehen und dadurch die genutzte Kommunikation zwischen Geräten wesentlich beeinflusst.

Beim Muster der Cloud-zentrierten Architektur kommunizieren Geräte nie direkt miteinander, sondern über die Cloud als Mittelsmann. Datenhaltung und Automationen liegen serverseitig außerhalb des mit Smarttechnik ausgestatteten Gebäudes in Anbieterhand. Jegliche Kommunikation zwischen Geräten erfolgt über diesen Server.

Vorteil des Ansatzes ist in erster Linie die Auslagerung der Rechenleistung. Es muss vor Ort weder ein Server installiert, noch gewartet oder mit Strom versorgt werden. Auch können Feature und Sicherheitsupdates für die lokal installierten Geräte einfach durch den Hersteller ausgerollt werden und verlangen keine technischen Kompetenzen vom Nutzer. Als Nachteile sind in erster Linie Datenschutzbedenken zu nennen, die besonders bei Smart-Geräten mit Fähigkeit zur Ton- oder Videoaufnahme schwer wiegen. Auch die Abhängigkeit vom Internet beziehungsweise der generellen Erreichbarkeit des Anbieterservers sind zu nennen, ebenso wie die erhöhte Latenz, die entsteht, indem alle Daten in das Internet gesendet werden.

Im Gegensatz zur Cloud-basierten Architektur steht die lokale Architektur. Alle Controller oder Servern, auch Hubs genannt, befinden sich im vor Ort, im lokalen Netz. Daten, beispielsweise Voreinstellungen, Szenen, Automationen, werden dort gespeichert. Die Smart-Home-Geräte kommunizieren - sofern nicht anders konfiguriert - überhaupt nicht über das Internet. Die Folge ist ein verbesserter Datenschutz. Auch die Latenz sinkt durch die maßgeblich kürzeren Kommunikationswege. Eine Störung der Internetverbindung hat keine Auswirkung auf die

Funktionalität des Smart-Homes. Nachteilig zu bewerten ist hier der bedeutend höherer Einrichtungs- und Administrationaufwand. Eine gewisse technische Affinität muss beim Nutzer bestehen.

Am häufigsten verbreitet ist die dritte Form der Architektur, ein Hybrid aus Cloud- und Lokalansatz, der Zuverlässigkeit mit Komfort kombiniert. Zeitkritische Steuerung und weniger rechenintensive Datenverarbeitung werden auf einem lokalen Controller ausgeführt, der seinerseits mit der Cloud verbunden ist. Durch diese Anbindung werden automatische Updates, Fernzugriff und das Speichern großer Datenmengen ermöglicht. Dieser Ansatz behebt viele der Nachteile beider anderer Ansätze. Datenschutzrisiken bestehen allerdings weiter.

Unter den verbreiteten Ökosystemen sind kaum Beispiele für den Cloud-exklusiven Ansatz zu finden. Hierin fallen vor allem Einzelgeräte mit Companion-App des Herstellers, wie smarte Glühbirnen, LED-Streifen oder Küchengeräte. Häufig sind diese jedoch in die großen Ökosystem integrierbar. Diese sind im wesentlichen Alexa von Amazon, HomeKit von Apple, Google Home von Google und SmartThings von Samsung alle vier sind der hybriden Architektur zuzuordnen. Als Vertreter des lokalen Ansatzes ist vor allem die als OpenSource verfügbare Software Home Assistent zu nennen, die durch eine Vielzahl an Plugins alle gängigen Kommunikationsprotokolle und damit Geräte beinahe jedes Herstellers unterstützt.

Da der zu entwickelnde Prototyp auch als Demonstrator für die Kommunikation über verschiedene Protokolle hinaus dienen soll, wird unter Verwendung von HomeAssistent weiter verfahren werden.

1.2. Kommunikationsmodelle in Smart-Home-Systemen

Wie bei der Architektur des System ist auch zwischen den theoretischen Modellen der Kommunikation zwischen Geräten zu unterscheiden.

Gängig ist das klassische Response/Request Pattern, bei dem per HTTP- oder REST-Request der aktuelle Stand eines Geräts abgefragt werden kann. Das Gerät beantwortet den Request im selben Protokoll. Dieser Ansatz eignet sich für die Steuerung von Geräten, die über wenige verschiedene Status verfügen oder nur selten geändert werden.

Ebenfalls weit verbreitet ist das Publish/Subscribe Pattern, das beispielsweise vom Mosquitto-Protokoll (MQTT) verwendet wird. Sender und Empfänger werden über Topics entkoppelt. Geräte, die einem Topic subscribed sind, erhalten alle Nachrichten die in selbigen gepublished werden. Das eignet sich besonders für große verteilte System mit vielen Klienten und großer Ereignisdichte.

Dezentrale Ansätze existieren ebenfalls und eignen sich besonders für die kabellose Kommunikation. Hier muss ein Gerät nicht zwangsläufig vom Hub aus erreichbar sein. Andere Geräte bilden die Brücke für Kommunikation, indem Nachrichten weitergegeben werden.

Unabhängig vom Kommunikationsmodell kann die Verwendung von digital Twins die Robustheit eines Systems verbessern. Der Zustand eines physischen Geräts wird dabei von einem digitalen Modell repräsentiert. Befehle des Hubs ändern den Zustand des digitalen Zwillings, der dem Soll-Zustand des Geräts entspricht. Das physische Gerät liefert den Ist-Zustand und handelt die Angleichung mit dem digital Twin aus. Das hat den deutlichen Vorteil, dass der Hub die Statusänderung sofort als gelungen annehmen kann und sich nicht mit der möglicherweise langwierigeren Kommunikation befassen muss.

1.3. Verwendete Protokolle

Das IP-basierten HTTP beziehungsweise HTTPS Protokoll bietet eine einfache und universelle Möglichkeit für die Netzwerkkommunikation, ist aber bei einer Vielzahl an Geräten oder vielen Anfragen nicht immer ideal. Das ebenfalls auf IP-Basis basierende WebSocket eignet sich schon besser für die bidirektionale Kommunikation großer Datenmengen und erlaubt es nach einmaligem Verbindungsaufbau, große Datenmengen hin und her zu streamen. Bereits erwähnt wurde das MQTT-Protokoll, eine leichtgewichtige Implementation des Publish/Subscribe-Patterns. MQTT bildet das Rückgrat vieler Internet-of-Things-Systeme.

Im Funkbereiche Dominien neben WiFi und dem niedrig-frequenteren und dadurch stromsparenderen Bluetooth die Protokolle Zigbee und Z-Wave, wobei letzteres eine proprietäre Implementation des mesh-fähigen Protokolls ist, das besonders bei kabellosen Sensoren und Schaltern angewandt wird.

Grundlage aller IoT-Protokolle bildet eine Transportverschlüsselung mit gegenseitiger Authentifizierung mittels Zertifikaten und Schlüsselpaaren, die regelmäßig rotieren, um keine Angriffsvektoren zu bieten.

Neu ist der von Apple vorangetrieben Interoperabilitätsstandard Matter, der auf IP-Basis über Ethernet oder WiFi ein einheitliches Onboarding und standardisierte Geräteklassen erlaubt um so die Kompatibilität zwischen Plattformen zu erweitern. Eine Vielzahl an Geräten ist heute mutterkompatibel, sodass kaum noch Plattformexklusivität zu finden ist. Aus diesen Gründen bietet Matter sich hervorragend für den Prototypen an.

1.4 Trends im IoT

Neben dem relativ neuen Matter und der Verwendung von digital Twins, sind generelle Trends der Branche auch im Smart-Home-Bereich wieder zu erkennen. Durch die steigende Zahl an Haushalten mit lokalen Batteriespeichen in Verbindung mit PV-Anlagen steigt der Bedarf, das Haus als ganzes mit Hilfe von Smarthome energieeffizient zu gestalten, indem beispielsweise Automationen an Sonnenstunden und Ladestand gekoppelt werden, oder der Ladezeitpunkt eines Elektro-Autos durch das Smart-Home-System dynamisch bestimmt werden soll. Eine, durch die Adaption der Technik von der breiten Masse ausgelöste, stärkere Wahrnehmung der Sicherheitsrisiken, die das Internet of Things birgt, führt zu stetig steigenden Ansprüchen sowie

besseren Implementation sicherer Updateketten und robusterer Kommunikation, was sich auch im Matterstandard widerspiegelt. Zuletzt ist natürlich auch der KI-Boom ein Faktor. EdgeAI und TinyML sind Trends die minimale, effiziente, häufig lokale künstliche Intelligenz auf zunehmend kleineren Geräten, damit insbesondere Smart-Home-Hubs und -Clients versprechen.

2. Abgeleitete Anforderungen

Aus den gesammelten Beobachtungen können wir einige Anforderungen für den zu entwickelnden Prototypen ableiten:

1. Das System soll verteilt aufgebaut sein, mit verschiedenen System in klaren Rollen, die miteinander kommunizieren um Steuerung und Automatisierung zu ermöglichen.
2. Es muss skalierbar sein, neue Geräte sollen bei Bedarf hinzugefügt werden können.
3. Es soll robust und fehlertolerant sein, ein Teilausfall soll den Rest des Netzes nicht beeinflussen, eine einmalige Kommunikationsstörung zwischen Hub und Gerät soll keine spürbaren Auswirkungen nach sich ziehen.
4. Das System soll Heterogenität von Geräten erlauben. Es sollen mehrere Protokolle und Dienste unterstützt werden.
5. Es soll sicher sein. Kommunikation soll Ende-zu-Ende-verschlüsselt sein, sensible Daten lokal verarbeiten sowie datensparsam und nachvollziehbar arbeiten.

2.1 gewählte Technologien für den Prototypen

Aus den Aspekten der Sicherheit und Heterogenität leitet sich HomeAssistent als sinnvolle Hubsoftware ab. Die restlichen Anforderungen lassen sich durch die Verwendung des Matterprotokolls verwirklichen. Um die Heterogenität zu zeigen, werden zusätzliche einige virtuellen Geräte mit MQTT angebunden.

3. Aufbau des Prototyp

Home Assistent ist mit Docker einfach über das offizielle Image installierbar, ähnlich verhält es sich mit dem Matterserver, der die Schnittstelle für die Kommunikation zwischen HomeAssistent und den Geräten bilden wird. Um zusätzlich MQTT-Geräte unterstützen zu können, wird auf gleiche Art der MQTT-Broker von offizieller Stelle installiert.

Nach dem Start mit Docker compose und problemfreier Einrichtung in der Weboberfläche von HomeAssistent ist das System startklar. Hätten wir bestehende Geräte, könnten diese einfach über das Menü hinzugefügt werden.

Das das nicht der Fall ist, hilft GitHub Copilot mit der Kreation von virtuellen Smart-Home-Geräten als jeweils separate Docker-Container aus, was einfach und problemfrei zu bewerkstelligen ist. In Zusammenarbeit mit der KI entsteht zudem für jedes virtuelle Gerät eine Web-Oberfläche die, je

nach Rolle, den Status wiedergibt oder die Manipulation der Sensoren erlaubt. Im Fall der Matterkomponenten erfolgt das mit einer als HTML gekosteten Statusseite auf der ohnehin bestehenden Netzwerk Adresse des Containers, bei MQTT muss eine solche extra vergeben werden, obwohl sie für das Protokoll selbst eigentlich nicht nötig gewesen wäre.

Es ergeben sich folgende Komponenten:

- Eine Glühbirne,
- Ein RGB-fähiger LED-Streifen,
- Ein Kontaktsensor an einem virtuellen Fenster,
- Ein Bewegungsmelder der in HomeAssistant als Anwesenheitssensor geführt wird,
- Ein Heizungsthermostat.

Bis auf das Thermostat, das mittels MQTT kommuniziert handelt es sich um Matterkomponenten.

Für jedes Gerät auf Matterbasis wird beim starten ein eindeutiger Code generiert, der entweder als QR-Code gescannt, oder in unserem Fall in den Einrichtungsassistenten von HomeAssistant eingegeben wird. Wir wählen Namen und Raum aus und das Gerät ist betriebsbereit. Im Fall unseres MQTT-Thermostates läuft die Einrichtung noch einfacher. Sobald verbunden, fügt der MQTT-Broker das Gerät automatisch hinzu. Name und Raum können hier natürlich nachträglich gewählt werden.

Ebenfalls in HomeAssistant konfigurieren wir nun zu Demozwecken einige Routinen. Wird das Fenster geöffnet, schaltet das Thermostat ab; meldet der Bewegungssensor einen leeren Raum, schaltet das Licht aus.

Alles läuft reibungslos, auch die Steuerung der Geräte über die Web-UI. So lassen sich die Temperatur der Heizung anpassen, die Lichter schalten und die Farbe der RGB-LED steuern. Auch nach einem simulierten Stromausfall, dem Neustart des Dockers, läuft wieder alles wie gehabt.

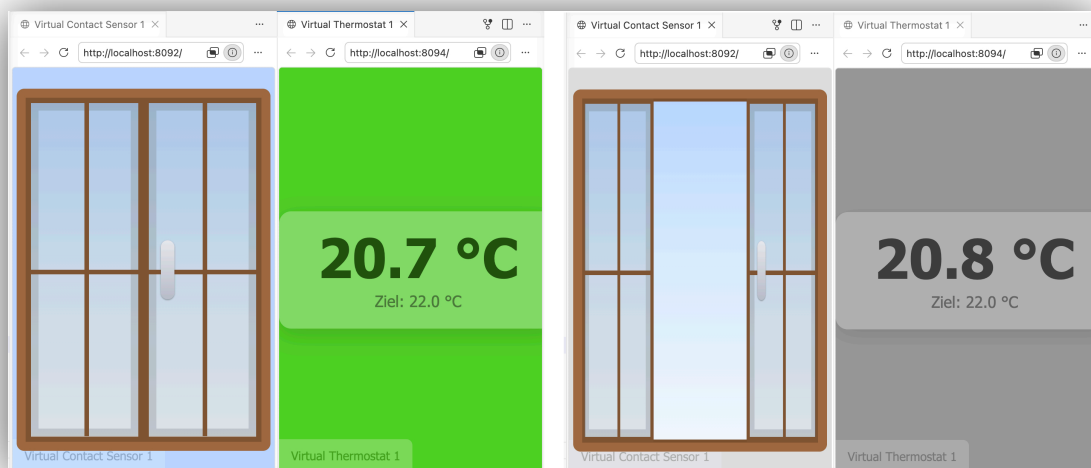


Abb. 1 Öffnet man das virtuelle Fenster mit einem Klick auf den Griff, schaltet das Thermostat ab.